International Journal of Information Security (2026) 25:109
https://doi.org/10.1007/s10207-026-01264-8

REGULAR CONTRIBUTION

Comparative evaluation of machine learning methods for protecting LLMs from prompt injection attacks

Nazarii Dzhaliuk[1] · Dmytro Sabodashko[1] · Volodymyr Khoma[1,2] · Yuriy Khoma[1] · Viktor Kolchenko[1] · Michal Podpora[3]

Received: 3 November 2025 / Accepted: 18 April 2026 / Published online: 15 June 2026
© The Author(s) 2026

Abstract

Prompt Injection attacks are a significant threat to the security of Large Language Models, giving adversaries the possibility to manipulate model outputs and bypass guardrails and restrictions. This study explores the effectiveness of selected Machine Learning -based approaches in detecting such attacks, comparing four classes of general-purpose and specialized defense approaches. Our empirical evaluation helps to identify and discuss each method, showing that detection effectiveness is strongly dependent on detection strategies and deployment trade-offs in inline defenses. Using a large benchmark of over 300k labeled prompts, selected following a comparison with three additional open-source corpora, we evaluate (1) traditional ML-based classifiers using embeddings, (2) independently fine-tuned encoder-based transformer models (DistilBERT, RoBERTa, DeBERTa), (3) specialized transformer-based detection models pre-trained for Prompt Injection, and (4) general-purpose LLMs configured as classifiers. Our experiments show that fine-tuned models achieve the highest detection rate on balanced benchmarks, traditional ML-based classifiers provide reasonably strong accuracy while requiring less computation, specialized detector models obtain lower recall, and general-purpose LLM-based classifiers can provide competitive performance, depending on the model used. Our research contributes to improving the robustness of AI-driven security systems against Prompt Injection by presenting a comparative study that can help in selecting and deploying ML-based defense against Prompt Injection attacks.

Keywords Prompt Injection attacks · Large Language Models · Machine Learning for security · Adversarial input detection · Transformer models · Security evaluation

1 Introduction

It is important to begin with the clarification of the definition of Prompt Injection as provided by OWASP community [1]. According to OWASP, Prompt Injection refers to the manipulation of a Large Language Model (LLM) through specifically crafted input to cause the model to execute actions or produce outputs that were not originally intended by its developers [2]. This vulnerability becomes useful for the adversary, when they introduce (either directly or indirectly) manipulated input data, effectively "jailbreaking" the system's interpreter. Such manipulations can lead to critical issues, including data leakage, social engineering, and a range of other security and social consequences.

Direct injection, often used as a primary attack vector for jailbreaking, occurs when an adversary overwrites, exposes, or maliciously expands the underlying system prompt. This approach allows for exploiting internal systems by interacting with unsecured functionalities and data repositories accessible through the LLM, resulting in information leaks and other consequences.

Indirect injection occurs when the source of the modified information is not within the prompt, but when LLM receives it from external sources that may be under an attacker's control, such as websites, files, emails, or information provided for the RAG-based (Retrieval Augmented Generation) sys-

✉ Nazarii Dzhaliuk
nazarii.o.dzhaliuk@lpnu.ua

[1] Institute of Computer Technologies, Automation and Metrology, Lviv Polytechnic National University, Lviv, Ukraine

[2] Institute of Control Engineering, Opole University of Technology, Opole, Poland

[3] Institute of Computer Science, University of Opole, Opole, Poland

tems. In this scenario, an adversary can embed a malicious injection within external content, resulting in hijacking of the context of all conversations, not just the current session. Indirect Prompt Injection does not necessarily need to be visible or human-readable, if only the text is processed by the LLM.

In advanced attacks, an LLM can be manipulated to impersonate a malicious actor or interact with user-configured plugins. In these scenarios, the compromised LLM effectively assists the adversary by circumventing standard security measures, leaving the user unaware of the intrusion. Consequently, the compromised LLM acts as an agent for the attacker, enabling them to achieve their objectives without triggering typical protective mechanisms or alerting the end user about a possible incident.

1.1 Aim of the research

The main goal of this research is to evaluate the effectiveness of selected promising Machine Learning models in detecting Prompt Injection attacks. The study includes comparative empirical evaluation, performed on a single primary benchmark dataset selected from several reviewed publicly available datasets, in order to help to identify the most reliable strategies for mitigating security risks associated with prompt manipulation.

2 Related works

The Large Language Models' security challenges are both timely and significant research areas, with the Prompt Injection attacks becoming possibly the most significant threat to deployed LLM-based systems. This chapter is dedicated to the discussion of the existing research literature areas meaningful for our comparative evaluation of Machine Learning methods for Prompt Injection detection.

2.1 Large language model security vulnerabilities

The typical security challenges recognized for Large Language Models have been systematically cataloged by the *OWASP Top 10 for Large Language Model Applications* framework [3], which identifies the Prompt Injection as the most significant vulnerability class. This has been also universally adopted in industry security practices [4, 5], establishing Prompt Injection as a foundational security concern.

There are two basic types of Prompt Injection attacks: direct and indirect. Direct attacks involve malicious inputs provided directly to the model, typically as a part of the user prompt, and indirect attacks exploit external data sources that the model processes as the "injection point" of the malicious request [6, 7]. This classification is important for understanding and choosing appropriate detection and mitigation techniques, as indirect attacks present additional challenges due to their possible permanent presence in the retrieved resources (as opposed to one-session-only breaches) and/or potential for delayed activation.

The relationship between Prompt Injection and broader jailbreaking attacks has been explored extensively [8, 9]. While jailbreaking typically aims to elicit harmful content from aligned models, Prompt Injection focuses on manipulating model's output to bypass application-level restrictions.

Recent work has highlighted the persistence of these vulnerabilities, even in safety-aligned models [10]. Token-level analysis of transformer architectures suggests fundamental limitations in alignment approaches [10], while empirical studies exactly demonstrate how fine-tuning can compromise safety measures [11, 12]. These findings emphasize the necessity of additional external detection mechanisms to reinforce model-level defenses.

2.2 Machine learning approaches for adversarial text detection

The adversarial text detection is a broad field of research/security, providing foundations for the below- mentioned Prompt Injection detection methods. Adversarial attacks in NLP (Natural Language Processing) include multiple approaches, of many complexity levels: from character-level manipulations to semantic modifications [13, 14]. These attack methodologies help define the types of linguistic manipulations that detection systems should be able to recognize.

Character-level attacks on transformer models, which are often used as e.g. hate speech classifiers, may serve as a good example to demonstrate both the vulnerability of these systems and potential defense mechanisms [15]. This study shows that achieving adversarial robustness in transformers requires specialized training procedures and architectural adjustments, which have direct implications on the design of detection models for Prompt Injection.

Another promising approach type is by using embedding-based detection. This detection concept has been implemented across various adversarial scenarios [16] with success. The core insight (that adversarial text often includes distinctive patterns in high-dimensional embedding spaces) is a foundation of many Prompt Injection detection methods. However, the effectiveness of these detection methods is strongly dependent on the quality and representativeness of the embedding space.

Semantic smoothing techniques are considered to be a complementary defense strategy [17]. By introducing controlled noise into inputs and analyzing output consistency, these methods can identify inputs that produce unstable or manipulated responses. Although they are relatively com-

putationally expensive, the semantic smoothing gives the possibility to assess the robustness characteristics that detection systems should have.

2.3 Transformer-based models for security applications

The application of transformer architectures to cybersecurity tasks became a relatively popular approach, giving promising results for systems for Prompt Injection detection. Transformer models are being successfully used in malware detection, network anomaly identification, and vulnerability assessment [18, 19]. These applications help to see the possible usability of transformer-based approaches for security classification tasks.

BERT and its variants have been particularly successful in security-related text classification tasks. Exemplary applications include malicious URL detection [20], phishing email identification, and malicious code analysis [21]. The success of these applications suggests that pre-trained language models can effectively learn security-relevant patterns from textual data.

The finetuning process for security applications presents unique challenges. Recent research shows that standard finetuning can compromise safety alignment of language models [22], creating potential vulnerabilities in detection systems themselves. This emphasizes the importance of careful evaluation of pre-trained models if they are adapted for security tasks.

In security contexts, transformer models should be designed to handle adversarial inputs robustly and produce interpretable result. Security applications should operate with high precision to minimize false positives, and to maintain recall sufficient enough to detect novel attack patterns.

2.4 Prompt injection detection methods

Specialized detection methods for Prompt Injection were being researched not only by the academic community, but also within commercial security products. Early approaches were focused on embedding-based classification, basing on the observation that malicious prompts often have specific patterns within semantic embedding spaces [23].

The ProtectAI models represent an important commercial development in the area. Their DeBERTa-based detection models [24], trained specifically for Prompt Injection detection, are often used as a baseline for comparison of other methods. They demonstrate that specialized transformer approaches can be effective, especially when trained using domain-specific data.

Recent literature explores also hybrid detection approaches, merging heuristic features with Deep Learning models [25], in order to capture surface-level indicators, and semantic patterns typical for malicious prompts. According to the authors, no single detection method is universally effective across all attack types, which might be a reason for the better performance of hybrid approach.

Attention-based detection approaches are another promising direction [26]. By analyzing attention patterns in target models, these approaches can identify when input prompts cause unusual attention distributions that may indicate injection attempts. However, these methods require access to model internals, limiting their applicability in practical scenarios.

2.5 Comparative evaluation frameworks

The evaluation of Prompt Injection detection systems brings numerous methodological challenges. Opposed to traditional text classification tasks, Prompt Injection detection is expected to detect and assess adversarial inputs designed specifically to evade detection. This requirement leads to the need for development of specialized evaluation frameworks and datasets.

Benchmarking methodologies for jailbreaking and Prompt Injection attacks have evolved to cope with the dynamic nature of these attack vectors [8, 9]. The frameworks emphasize the importance of the detection of diverse attack types, in realistic deployment scenarios, and with precise evaluation. At the same time, the creativity and rapid evolution of attack techniques may cause the static benchmarks to become obsolete.

Dataset balance is an interesting and important aspect in any research, but in Prompt Injection detection-related datasets turns out to be particularly critical. The rarity of actual attacks in production systems is a really significant class imbalance challenge [27]. This imbalance affects training and evaluation, requiring proper consideration of sampling strategies and performance metrics.

The trade-offs between detection approaches involve: accuracy vs computational cost, false positive rates vs coverage of novel attacks, and interpretability vs performance. These challenging decisions are also a subject of research and have begun to be systematically explored [17], but comprehensive evaluations across different model families and deployment scenarios are still very hard to come by in research literature.

2.6 Research gap and contribution context

Many individual detection approaches have been explored, but they were usually presented in isolation, without a deeper comprehensive comparative evaluation. Existing studies typically focus either on single methods or on narrow comparisons within a "family" of similar approaches. This limitation makes it difficult for practitioners to make informed decisions

about detection system deployment and for researchers to investigate strengths and weaknesses of different approaches.

The lack of systematic evaluation across traditional Machine Learning methods, fine-tuned transformers, specialized security models, and the idea of employing general-purpose LLMs to cybersecurity analysis [28] represents a noticeable gap in the literature. Each approach category brings distinct advantages and limitations, but these trade-offs have not been systematically compared.

Our work is intended to address this gap, by providing a comprehensive empirical comparison across four major detection paradigms, using a single selected benchmark dataset and standard evaluation metrics to characterize performance trade-offs. This comparative approach builds on the work in adversarial text detection, addressing the challenges and requirements of Prompt Injection detection in production environments.

3 Methods and materials

To conduct a comprehensive evaluation of Prompt Injection detection approaches in Large Language Models (LLMs), the authors designed an experimental benchmarking framework (implementation available at [29]). This chapter presents the methodological choices of the study, including dataset selection, and model configurations.

The methodology (both: the chapter and our research activity) is structured to enable a comparative analysis across four distinct categories of defense approaches: (1) traditional Machine Learning classifiers using embeddings, (2) transformer-based models finetuned using labeled data, (3) specialized models, pre-trained specifically for Prompt Injection detection, and (4) general-purpose LLMs employed as zero-shot or instruction-tuned classifiers. Each category implies its unique trade-off between precision, recall, computational cost, and ease of pipeline integration.

The first task was to find and assess publicly available Prompt Injection datasets and evaluate them based on volume, label balance, and their applicability to real-world scenarios. From this analysis, a primary dataset was selected for training and testing. Subsequently, we prepared multiple experimental pipelines to evaluate classification performance using established metrics: accuracy, precision, recall, and F1-score.

This framework was designed to enable a comparative evaluation of detection strategies under varying conditions, giving insights into how to defense against possible Prompt Injection attacks in production systems and pipelines.

The following subsections describe in detail the datasets used, classification models implemented, and experimental procedures adopted for our study.

3.1 Dataset overview

Selecting an appropriate dataset for training and testing models in Prompt Injection detection is a factor that directly impacts the effectiveness of the final systems. A high-quality dataset should give the model representative examples of both legitimate queries and diverse examples of Prompt Injection attacks.

A well-structured dataset should include a wide range of various attack types, including direct prompt modifications, hidden instructions, role-playing exploits, and other adversarial strategies. Additionally, it should include a strong variety of natural language patterns to prevent the model from overfitting to specific phrasing styles, and to maintain high generalization capabilities.

To make sure that the dataset remains robust and applicable in real-world scenarios, it is important to be able to update it periodically with new attack variations. This dynamic approach is important in order to keep the models up-to-date to emerging threats and new Prompt Injection techniques.

As part of this study, several datasets were analyzed based on their quality and suitability for solving the given task:

1. **Harelix/Prompt-Injection-Mixed-Techniques-2024** [30] – a dataset used for training specialized models in Prompt Injection detection.
   - Total records: 1,165
   - Classification categories: benign, malicious
   - Train–test split: training data only
   - Training distribution: 874 malicious (74.4%), 291 benign (24.8%)
2. **jackhhao/jailbreak-classification** [31] – a dataset utilized by ProtectAI for fine-tuning the microsoft/ deberta-v3-base model to detect malicious prompts.
   - Total records: 1,306
   - Classification categories: benign, malicious
   - Train–test split: available
   - Training distribution: 527 malicious (50%), 517 benign (50%)
   - Testing distribution: 139 malicious (53%), 123 benign (46%)
3. **jayavibhav/prompt-injection** [32] – a large-scale dataset used for research in Prompt Injection detection.
   - Total records: 327,400
   - Classification categories: 0 (benign), 1 (malicious)
   - Train–test split: available
   - Training distribution: 128,380 malicious (49%), 146,720 benign (50.6%)
   - Testing distribution: 32,046 malicious (50.6%), 33,092 benign (49.4%)

4. **xTRam1/safe-guard-prompt-injection** [33] – a relatively large dataset utilized for Prompt Injection classification research.
   - Total records: 10,300
   - Classification categories: 0 (benign), 1 (malicious)
   - Train–test split: available
   - Training distribution: 2,472 malicious (30%), 5,685 benign (69%)
   - Testing distribution: 659 malicious (32%), 1,400 benign (68%)

Each dataset presents unique characteristics and label distributions, which influence the model's ability to generalize and effectively detect Prompt Injection attacks (Table 1).

A comparative analysis of the four datasets reveals that the first dataset does not meet the requirement for a clear train-test split, making it unsuitable for this study. In contrast, datasets #2, #3, and #4 adhere to this criterion. After verifying all selection criteria, the next step involved comparing the total number of queries and their distribution across training and testing subsets. Among the candidates, dataset #3 (jayavibhav/prompt-injection) was considered to be the most suitable candidate. This dataset contains 327,400 total queries, making it the largest among the four. Additionally, both malicious and benign (non-malicious) queries are nearly evenly distributed within the training and testing subsets. The balanced nature of the dataset, as reflected in the percentage of each class, ensures a more representative learning process for the model. Thus, the jayavibhav/prompt-injection dataset was selected as the primary dataset for further research. It should be acknowledged, however, that relying on a single primary dataset is a limitation of this study. Its specific attack distributions and linguistic patterns may not fully represent the diversity of real-world Prompt Injection attempts, and findings may not generalize to corpora with different characteristics.

3.2 Approaches for protecting large language models from prompt injection attacks

One of the primary approaches involves validating and sanitizing input data before it is passed to the model. This process includes filtering out potentially dangerous character sequences, removing special markers, and escape codes symbols. Additionally, it is crucial to limit the length of input queries and to employ tokenization mechanisms that reduce the opportunities for manipulation. However, relying solely on statistical methods makes it challenging to accurately interpret context and determine whether an input constitutes a Prompt Injection that could lead to inappropriate behavior by the model.

Another effective approach involves implementing a system of role-based constraints (the model should not "take roles" or engage in any "playing" or "pretending") and explicit instructions for the model. The model should have clearly defined behavioral boundaries and must adhere to these limits even when faced with manipulative prompts. It is also very useful to include anomaly detection mechanisms that can report potential attacks (even if these would be simple pattern analysis mechanisms executed on the prompts).

As a result of the literature research presented in chapter 2, we have chosen to focus on the following four approaches:

1. Traditional Machine Learning Methods; Techniques such as Random Forest, Support Vector Machines (SVM), and gradient boosting, enabling the development of baseline classifiers for detecting attacks.
2. Fine-Tuning Pre-Trained Models; Adapting pre-trained models to the specific nuances of the task, achieving high accuracy in detection.
3. Specialized Models; Models specifically optimized for the detection of Prompt Injection attacks.
4. General-Purpose Large Language Models: Employing LLMs as NLP-based analysis tools [28] to try to identify Prompt Injection attempts.

Validation and verification of the results were conducted using standardized classification quality metrics described in the previous section, namely:

- Accuracy (classification correctness)
- Precision
- Recall
- F1-score (composite metric)

The accuracy and objectivity of the statistical analysis of the experimental data was ensured by using the standard sklearn Python library's metrics module.

3.3 Models used in experiments

Three transformer models with distinct architectural and computational characteristics were selected for the study: DistilBERT, RoBERTa, and DeBERTa. Each of these models represents a unique approach to processing linguistic information and has meaningful differences in parameter count, computational speed, and semantic analysis accuracy.

In our study, the DistilBERT model is implemented using the base model from the HuggingFace platform: distilbert/distilbert-base-uncased-finetuned-sst-2-english. It belongs to the class of lightweight models with optimized architecture that minimizes computational costs, thus maintaining an acceptable level of classification quality. The model comprises 66 million parameters and is known for its

Table 1 Prompt Injection datasets considered within this study

No	name	size	classification	division	train		test	
					0	1	0	1
1.	Harelix/Prompt-Injection-Mixed-Techniques-2024	1 165	binary – – 0 or 1	no – only for testing	291 (24.8%)	874 (74.4%)	–	–
2.	jackhhao/jailbreak-classification	1 306	binary – – 0 or 1	yes – training and testing	517 (50%)	527 (50%)	123 (46%)	139 (53%)
3.	jayavibhav/prompt-injection	327 400	binary – – 0 or 1	yes – training and testing	146 720 (50.6%)	128 380 (49%)	33 092 (49.4%)	32 046 (50.6%)
4.	xTRam1/safe-guard-prompt-injection	10 300	binary – – 0 or 1	yes – training and testing	5 685 (69%)	2 472 (30%)	1 400 (68%)	659 (32%)

high processing speed and relatively reliable baseline accuracy in semantic analysis.

RoBERTa is implemented using the model version distilbert/distilroberta-base. Developed by Facebook AI research team, demonstrates better resilience to variability in input data, thanks to an improved pre-training methodology. With 125 million parameters, the model offers high semantic decoding accuracy and efficient contextual analysis.

DeBERTa is implemented using the microsoft/deberta-v3-base. Created by Microsoft Research, it represents the most complex architecture among the models under study, comprising 180 million parameters. Its unique mechanism for contextual decoding, which uses both absolute and relative positional embeddings, is expected to provide the highest level of semantic understanding among the examined models.

3.4 Classification method using traditional machine learning

In the context of the conducted research, a comprehensive experimental analysis methodology was implemented using leading classification algorithms. The research process included the following methodological steps:

- Data preparation - A specialized dataset, jayavibhav/prompt-injection, was used to ensure the representativeness of the study. An important step of our methodology was the generation of vector representations (embeddings) for queries from both the test and training datasets using each of the used models.
- Experimental Implementation - The experimental research was conducted using four classical Machine Learning algorithms:
  - Naive Bayes: Implemented via GaussianNB from the sklearn.naive_bayes module.
  - LGB Classifier: Implemented using lgb.LGBMClassifier from the LightGBM library.
  - Random Forest: Implemented as RandomForestClassifier from the sklearn.ensemble module.
  - MLPClassifier: Implemented using MLPClassifier from the sklearn.neural_network module.

Table 2 Classification performance of traditional Machine Learning methods for Prompt Injection detection [%]

	Acc.	Prec.	Recall	F1
Naive Bayes	60.04	57.52	72.86	64.28
LGB Classifier	90.42	87.18	94.48	90.68
Random Forest	83.07	78.00	91.51	84.22
MLP Classifier	**96.80**	**98.60**	**94.85**	**96.69**

This approach turned out to be successful in allowing for a robust comparative analysis of the performance of these classifiers in Prompt Injection attacks detection.

The methodological approach involved the sequential testing of each classification algorithm on the preprocessed vector representations of the data. This resulted in a comprehensive evaluation of the effectiveness of traditional Machine Learning methods in detecting Prompt Injection attacks, presented in Table 2.

The classification study performed with the use of traditional Machine Learning methods shows that the model microsoft/deberta-v3-base exhibited the highest performance in most cases. Its superior performance can be attributed to its deeper architecture and more advanced text vectorization. The distilroberta-base model also gave high results, especially when combined with the MLPClassifier, although it was consistently lower than deberta-v3-base in overall metrics. As anticipated, the distilbert-base model demonstrated the lowest performance possibly due to its simplified architecture.

Among the evaluated models, the combination of microsoft/deberta-v3-base with the MLPClassifier proved to be the optimal solution for text classification tasks, achieving

Table 3 Classification performance of fine-tuned transformer models for Prompt Injection detection [%]

Model	Acc.	Prec.	Recall	F1
DistilBERT	99.42	99.39	99.43	99.41
DistilRoBERTa	99.62	99.65	99.58	99.61
DeBERTa	**99.58**	**99.83**	**99.32**	**99.57**

the highest scores across all evaluated metrics. These results clearly show the advantage of employing more advanced and complex modern transformer architectures, such as DeBERTa, in conjunction with neural networks -based MLP-Classifier for Natural Language Processing applications.

3.5 Classification using independently fine-tuned models

This part of our study presents comparative analysis of detecting Prompt Injections through classification using independently fine-tuned models. The most important aspects of the research include: adapting pre-trained models to the specific challenges of Prompt Injection detection, using fine-tuning procedures to enhance classification accuracy, and analyzing the effectiveness of various architectures in identifying manipulative inputs.

The methodology included sequential evaluation of the fine-tuned models' capabilities to detect potentially dangerous textual inputs. A specialized dataset, jayavibhav/prompt-injection, was employed as the foundation for assembling a representative collection of experimental materials and training data for the three selected models: distilbert/distilbert-base-uncased-finetuned-sst-2-english,
distilbert/distilroberta-base, and microsoft/deberta-v3-base.

Upon loading the specified dataset, it was divided into two subsets: one for training and one for testing in an 80/20 ratio. Specifically, 80% of the data was used for the model's training process, while the remaining 20% served as a validation part. Data partitioning ensures that the model is not trained using validation data, and it is not tested using the training data. This classical approach provides a more representative assessment of its performance after the fine-tuning process.

The model training was conducted using an epoch-based learning strategy. In the context of fine-tuning, the epoch-based approach involves iteratively processing the entire training dataset, with each complete pass through the data being referred to as an epoch. The model incrementally refines its weights by minimizing the loss function through repeated exposure to the data, achieving better generalization of the results.

After comparing the final results (Table 3) of the three models (microsoft/deberta-v3-base, distilbert-base-uncased-finetuned-sst-2-english, and distilroberta-base) interesting conclusions can be drawn regarding their effectiveness in detecting Prompt Injection attacks.

The microsoft/deberta-v3-base model had the highest performance across all primary metrics. It achieved an Accuracy of 99.58%, Precision of 99.83%, Recall of 99.32%, and an F1-score of 99.57%. These results indicate the model's ability to correctly distinguish positive and negative instances, thereby minimizing the occurrence of both false positives and false negatives.

The distilbert-base-uncased-finetuned-sst-2-english model also showed strong performance, though slightly behind the DeBERTa model. With an Accuracy 99.42%. and balanced Precision and Recall metrics reflected in its high F1-score, this model proves to be effective in classification. However, it may introduce a few false positives or false negatives, compared to microsoft/deberta-v3-base.

Similarly, the distilroberta-base model produced results close to those of the DistilBERT model, showing high levels of accuracy and F1-score. Its metrics suggest a well-balanced performance; however, its Recall is somewhat lower, indicating a slight deficiency in detecting relevant cases.

In summary, distilbert-base-uncased-finetuned-sst-2-english offers a favorable balance between accuracy and efficiency, making it a strong candidate for resource-constrained applications; the microsoft/deberta-v3-base model stands out as the most precise one, its superior Precision, Recall, and F1-score make it particularly suitable for applications where achieving the highest possible accuracy is more important than the model complexity.

3.6 Classification method using specialized models

Since traditional approaches using general-purpose language models have demonstrated high effectiveness, increasing attention is being directed toward the development of specialized models designed specifically to identify prompt-related attacks. These models are based on transformer architectures optimized for handling textual vulnerabilities, thereby enhancing their accuracy and robustness against manipulative techniques.

This section presents the classification results obtained using such specialized solutions, their apparent advantages as well as the challenges encountered during their implementation. The aim is to assess the capabilities of available specialized solutions in detecting prompt injections, compare their effectiveness against universal models, and evaluate them.
The following models are considered:

– Model acuvity/distilbert-base-uncased-prompt-injection-v0.1 - This model is built upon the lightweight distilbert/distilbert-base-uncased-finetuned-sst-2-english base

Table 4 Classification performance of specialized Prompt Injection detection models [%]

Model	Accuracy	Precision	Recall	F1 score
distilbert/distilbert-base-uncased-finetuned-sst-2-english	**60.67**	**85.94**	**24.32**	**37.91**
meta-llama/Prompt-Guard-86M	46.66	45.64	42.14	43.82
protectai/deberta-v3-base-prompt-injection-v2	55.71	80.44	13.59	23.25

model, whose capabilities were demonstrated in previous experimental studies.

- Model meta-llama/Prompt-Guard-86M - Developed by Meta, tthis is a lightweight classifier (86M backbone parameters) fine-tuned for prompt-injection and jailbreak detection, built on the mDeBERTa-v3-base base model from Microsoft.
- Model protectai/deberta-v3-base-prompt-injection-v2 - Offered by ProtectAI, a company specializing in the security of Large Language Models and their integration into modern applications, this model is based on the previously evaluated microsoft/deberta-v3-base.

Using the jayavibhav/prompt-injection dataset with an 80:20 split, our study assessed the detection capabilities of these commercial solutions for Prompt Injection attacks. The evaluation focused on their ability to accurately classify malicious inputs, assess their resistance to adversarial manipulation, and determine their suitability for deployment.

Table 4 presents the classification results for Prompt Injection using the distilbert-base-uncased-finetuned-sst-2-english model, evaluated as the first commercial model in this study. It demonstrated lower effectiveness compared to earlier models, achieving an Accuracy of only 60.67%, indicating its limited ability to correctly classify examples. Although it shows a relatively high Precision of 85.94%, meaning that most positive predictions are correct, its extremely low Recall of 24.32% suggests that a substantial number of relevant instances are being missed, resulting in an overall F1-score of just 37.91%.

The meta-llama/Prompt-Guard-86M model's performance was the lowest among the evaluated models. Its Accuracy was 46.66%, reflecting significant issues in correctly classifying examples, while its Precision of 45.64% indicates that fewer than half of the predictions are correct. Although its Recall of 42.14% is somewhat higher than that of the first model, the overall performance, as indicated by an F1-score of 43.82%, remains limited.

The last model, protectai/deberta-v3-base-prompt-injection-v2, produced mixed results. With an Accuracy of 55.71%, it performed better than the meta-llama/Prompt-Guard-86M but still not as good as the distilbert-base-uncased-finetuned-sst-2-english model. It obtained a fair Precision of 80.44%, yet its Recall is notably low at only 13.59%, reflecting a high number of missed relevant cases and resulting in an F1-score of merely 23.25%.

Overall, while the distilbert-base-uncased-finetuned-sst-2-english model resulted with the highest Accuracy and Precision among the three, its Recall is moderately low. The meta-llama/Prompt-Guard-86M model shows the poorest performance across all metrics, and although the protectai/deberta-v3-base-prompt-injection-v2 model achieves better Precision, its Recall is alarmingly low, which significantly undermines its overall practical production-grade applicability.

3.7 Classification method using general-purpose Large Language Models

Large Language Models, originally developed for a wide range of Natural Language Processing tasks, are often employed for text classification because of their versatility and ability to handle diverse types of NLP-related data. In this section, we investigate the capabilities of such general-purpose models in detecting Prompt Injection attacks, comparing their effectiveness with specialized solutions.

As in previous experiments, the jayavibhav/prompt-injection dataset serves as the basis for our evaluation. A dedicated test subset from this dataset was used to gather data on the performance of general-purpose LLMs in detecting Prompt Injection attacks.

The analysis of the LLM-based approach was executed using the OpenAI model gpt-5-nano. For the experimental study, we utilized the API provided by OpenAI. The model was prompted to respond in the required format using corrective prompts sent prior to each Prompt Injection detection query. These additional instructions helped standardize the model's output, ensuring consistent formatting.

The results of using a general-purpose Large Language Model (gpt-5-nano) for detecting Prompt Injection through prompt-based classification indicate competitive effectiveness. As shown in Table 5, gpt-5-nano achieved the stronger overall performance of the two evaluated general-purpose LLM-based classifiers, with an Accuracy of 84.18%, Precision of 95.34%, Recall of 71.45%, and F1-score of 81.68%. These results indicate a well-balanced trade-off between precision and recall, suggesting that recent LLM variants may provide a viable option for Prompt Injection detection.

Table 5 Classification performance of general-purpose Large Language Models used as classifiers [%]

Model	Acc.	Prec.	Recall	F1
gpt-5-nano	84.18	95.34	71.45	81.68
microsoft/phi-3-mini -4k-instruct	43.45	44.30	56.61	49.71

In contrast, microsoft/Phi-3-mini-4k-instruct achieved lower overall effectiveness, with an Accuracy of 43.45%, Precision of 44.30%, Recall of 56.61%, and F1-score of 49.71%.

The second evaluated model, microsoft/Phi-3-mini-4k-instruct, is a lightweight open-source model from Microsoft. In our experiments, it achieved lower overall effectiveness than gpt-5-nano, with an Accuracy of 43.45%, Precision of 44.30%, Recall of 56.61%, and F1-score of 49.71%.

While gpt-5-nano did not achieve highest possible Recall, its results indicate a balanced trade-off between Recall and Precision, with a substantially lower false-positive rate than earlier general-purpose LLM-based classifiers. On the other hand, microsoft/Phi-3-mini-4k-instruct offers only moderate performance on both measures. As a result, gpt-5-nano appears to be the more suitable of the two evaluated models for practical Prompt Injection detection, especially in scenarios where minimizing false positives is an important consideration.

3.8 Threats to validity

This section discusses potential limitations and validity concerns that should be considered when interpreting the findings of our study.

From the perspective of internal validity, the hyperparameter configurations for fine-tuned transformer models were selected based on commonly accepted practices in the literature. Their values are included in our replication code repository [29]. Alternative configurations, including different learning rates, batch sizes, or number of training epochs, may produce different results. The embedding generation process for traditional Machine Learning classifiers depends on the quality of the underlying transformer models, which may introduce additional variability into the classification pipeline.

Regarding external validity, our evaluation relies primarily on a single benchmark dataset (jayavibhav/prompt-injection). While this dataset was selected for its substantial size and balanced distribution, this single-dataset dependency limits the generalizability of our findings. Models that perform well on this corpus may fail to transfer to datasets with different attack taxonomies, linguistic styles, or label conventions. A comprehensive multi-dataset evaluation would be necessary to make stronger claims about real-world detection capability. Additionally, the balanced class distribution of approximately 50% malicious and 50% benign samples does not reflect production environments where Prompt Injection attacks are relatively rare, which may cause the reported precision and recall values to be overly optimistic in practice.

In terms of construct validity, the definition and labeling of prompt injection attacks within the dataset may not capture all possible attack variations. Novel or emerging attack techniques that differ substantially from those represented in the training data may evade detection by the evaluated models.

Concerning conclusion validity, the static nature of the evaluation does not account for adversarial adaptation, where attackers iteratively modify their techniques to evade detection mechanisms. In real-life deployment scenarios, detection systems face evolving threat landscape that no benchmark-based evaluation can fully simulate.

4 Conclusions

This study presents a comparative evaluation of selected Machine Learning methods for protecting Large Language Models against Prompt Injection attacks. By conducting systematic experiments on a curated dataset, four categories of defense approaches were assessed: (1) traditional supervised classifiers, (2) fine-tuned transformer models, (3) specialized detection models, and (4) general-purpose LLM-based classifiers.

It is important to note a key methodological distinction in our evaluation approach. The specialized detection models were evaluated without retraining on our benchmark dataset, as these solutions are marketed as production-ready tools for prompt injection detection. Our objective was to assess their out-of-the-box generalization capability, reflecting the practical deployment scenario that practitioners encounter when adopting such tools. In contrast, the fine-tuned transformer models require task-specific adaptation, and thus, they were trained on the training subset. All models across all four categories were evaluated on the identical test dataset, producing a fair and consistent basis for performance comparison.

Key conclusions include:

1. First, traditional Machine Learning classifiers such as Naive Bayes, LightGBM, Random Forest, and Multi-Layer Perceptron (MLP) have fast training times and low computational overhead; they achieve up to 96.69% F1-score in classification of malicious vs benign prompts; their simplicity and efficiency make them suitable for real-time monitoring in resource-constrained environments.
2. Second, all of the tested fine-tuned transformer models (DeBERTa, DistilBERT, RoBERTa) consistently outper-

formed all of the tested traditional ML-based methods in all of the standardized classification quality metrics; however, their higher training and inference costs suggest deployment trade-offs for latency-sensitive applications.
3. Third, specialized detection models (in our case: ProtectAI) showed limited recall and high false-negative rates, prompting the current immaturity of the off-the-shelf solutions for Prompt Injection; the observed gap indicates an urgent need for improved model architecture research or hybrid detection pipelines that would be able to combine rule-based filtering with Machine Learning.
4. Fourth, among the evaluated general-purpose LLM-based classifiers, gpt-5-nano achieved an Accuracy of 84.18%, Precision of 95.34%, Recall of 71.45%, and F1-score of 81.68%, indicating a comparatively strong and well-balanced detection profile. This suggests that newer LLM variants may represent a viable option for Prompt Injection detection, provided that inference latency requirements can be accommodated.

Overall, the presented comparative analysis research suggests that no single method can be considered universally dominant. Rather, the deployment context, including performance requirements, resource availability, and acceptable risk levels, should guide defense selection. The research results help in the understanding of Prompt Injection mitigation and may encourage further exploration of hybrid defense approaches, integrating fine-tuned transformer models with fast, traditional ML-based methods. It should be noted that the findings of this study are based on a single primary benchmark dataset, which limits the generalizability of the conclusions. Future work should therefore prioritize evaluation across multiple diverse corpora, covering a broader range of attack types and real-world class distributions, in order to validate and extend the comparative insights presented here.

Author Contributions N.D. and D.S. proposed the methodology; N.D. and V.K. performed the trials; D.S., V.K., and Y.K. supervised the research; N.D., D.S., V.K., and M.P. wrote the initial manuscript; N.D., M.P., and D.S. refined the reviewed manuscript.

Funding This research did not involve external funding.

Data Availability The replication package containing the code implemented for this research: https://github.com/toyhtowdomasydyt/llm-prompt-injection-detector

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. OWASP. Owasp top 10 for llm applications version 1.1 (2023). https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-2023-v1_1.pdf. Accessed: 2025-05-05
2. OWASP. OWASP LLM01: Prompt injection (2024). https://genai.owasp.org/llmrisk/llm01-prompt-injection/. Accessed: 2025-05-05
3. OWASP Foundation. OWASP Top 10 for Large Language Model Applications (2023). https://owasp.org/www-project-top-10-for-large-language-model-applications/. Accessed: October 2025
4. Lasso Security. OWASP Top 10 LLM Vulnerabilities & Security Checklist (2024). https://www.lasso.security/blog/owasp-top-10-llm-vulnerabilities-security-checklist. Accessed: October 2025
5. Tigera. LLM Security: Top 10 Risks and 5 Best Practices (2024). https://www.tigera.io/learn/guides/llm-security/. Accessed: October 2025
6. F. Perez, I. Ribeiro, Ignore previous prompt: Attack techniques for language models, arXiv preprint arXiv:2211.09527 (2022)
7. The Moonlight. [Literature Review] Can Indirect Prompt Injection Attacks be Detected and Removed (2025). https://www.themoonlight.io/en/review/can-indirect-prompt-injection-attacks-be-detected-and-removed. Accessed: October 2025
8. A. Zou, Z. Wang, J.Z. Kolter, M. Fredrikson, Universal and transferable adversarial attacks on aligned language models, arXiv preprint arXiv:2307.15043 (2023)
9. P. Chao, A. Robey, E. Dobriban, H. Hassani, G.J. Pappas, E. Wong, Jailbreaking black box large language models in twenty queries, arXiv preprint arXiv:2310.08419 (2023)
10. E. Hubinger, C. Denison, J. Mu, et al., Sleeper agents: Training deceptive llms that persist through safety training, arXiv preprint arXiv:2401.05566 (2024)
11. X. Qi, Y. Zeng, T. Xie, P.Y. Chen, R. Jia, P. Mittal, P. Henderson, Fine-tuning aligned language models compromises safety, even when users do not intend to!, arXiv preprint arXiv:2310.03693 (2023)
12. Cisco Security. Fine-Tuning LLMs Breaks Their Safety and Security Alignment (2024). https://blogs.cisco.com/security/fine-tuning-llms-breaks-their-safety-and-security-alignment. Accessed: October 2025
13. AntiSpoofing.org. Adversarial Attacks in Natural Language Processing (2024). https://antispoofing.org/adversarial-attacks-in-natural-language-processing/. Accessed: 2025-10-01
14. Zhang, W.E., Sheng, Q.Z., Alhazmi, A., Li, C.: Adversarial attacks on deep-learning models in natural language processing. ACM Comput. Surv. **53**(3), 1 (2020)
15. J. Risch, R. Krestel, in *Proceedings of the 1st Workshop on Benchmarking: Past, Present and Future* (2022), pp. 1–8
16. E. Mitchell, Y. Lee, A. Khazatsky, C.D. Manning, C. Finn, Detectgpt: Zero-shot machine-generated text detection using probability curvature, arXiv preprint arXiv:2301.11305 (2023)
17. A. Robey, E. Wong, H. Hassani, G.J. Pappas, Smoothllm: Defending large language models against jailbreaking attacks, arXiv preprint arXiv:2310.03684 (2023)

18. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Advances in Neural Information Processing Systems **30** (2017)
19. J. Devlin, M.W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, arXiv preprint arXiv:1810.04805 (2018)
20. Google Cloud Security. Training Transformers Models to Detect Malicious URLs (2024). https://cloud.google.com/blog/topics/threat-intelligence/training-transformers-for-cyber-security-tasks-malicious-url-prediction. Accessed: October 2025
21. J. Chen, Y. Liu, X. Xie, Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs, arXiv preprint arXiv:2409.09368 (2024)
22. X. Yang, X. Wang, Q. Zhang, L. Petzold, W.Y. Wang, X. Zhao, D. Lin, Shadow alignment: The ease of subverting safely-aligned language models, arXiv preprint arXiv:2310.02949 (2024)
23. R. Goodside. Prompt injection attacks against gpt-3. Blog post (2023). Available online
24. ProtectAI. protectai/deberta-v3-base-prompt-injection-v2 (2024). https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2. HuggingFace Model Hub, Accessed: October 2025
25. Y. Liu, R. Wang, X. Li, K. Chen, Detection method for prompt injection by integrating pre-trained model and heuristic feature engineering, arXiv preprint arXiv:2406.06384 (2024)
26. N. Wei, X. Huang, X. Wang, S. Zhang, M. Zhou, Attention tracker: Detecting prompt injection attacks in llms, arXiv preprint arXiv:2411.03348 (2024)
27. Japkowicz, N., Stephen, S.: The class imbalance problem: A systematic study. Intelligent Data Analysis **6**(5), 429 (2002)
28. Conceição, T., Cruz, N.: Evaluation of the maturity of llms in the cybersecurity domain: T. conceição, n. cruz. Int. J. Inf. Secur. **24**(5), 197 (2025)
29. N. Dzhaliuk. LLM prompt injection detector - code repository (2026). https://github.com/toyhtowdomasydyt/llm-prompt-injection-detector. Accessed: 2026-01-01
30. Harelix. Dataset harelix/prompt-injection-mixed-techniques-2024 (2024). https://huggingface.co/datasets/Harelix/Prompt-Injection-Mixed-Techniques-2024. Accessed: 2025-07-01
31. jackhhao. Dataset jackhhao/jailbreak-classification (2024). https://huggingface.co/datasets/jackhhao/jailbreak-classification. Accessed: 2025-09-30
32. jayavibhav. Dataset jayavibhav/prompt-injection (2024). https://huggingface.co/datasets/jayavibhav/prompt-injection. Accessed: 2025-09-30
33. xTRam1. Dataset xtram1/safe-guard-prompt-injection (2024). https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection. Accessed: 2025-09-30

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.